

The Complete DORA Manual

Hardware setup, vehicle connection, and a full technical reference for every tool in the suite.

This manual covers everything needed to get DORA running against a real vehicle — required hardware (OBD-II adapters, K-line/CAN/J2534 flash interfaces), installation, configuration, and the app's safety architecture — followed by a complete panel-by-panel reference covering every feature: live monitoring, the map editor, ROM tools, ECU flashing, AI analysis, dyno testing, diagnostics, calculators, and shop/business management.

Contents

PART I — Setup

- What You'll Need (Hardware)
- Installing DORA
- Configuration (.env)
- Connecting DORA to Your Vehicle
- The App Shell — Sidebar, Header & Shortcuts

PART II — Safety Architecture

- How the Safety Guard Works
- Hard Limits Reference

PART III — Complete Feature Reference

- MONITORING
- TUNING
- AI
- ANALYSIS
- ROM TOOLS
- ECU
- HARDWARE
- CALC
- REPORTS
- WORKFLOW
- BUSINESS
- SYSTEM

PART IV — Troubleshooting & Appendix

- Troubleshooting
- Keyboard Shortcuts Quick Reference

What You'll Need

DORA runs as a Windows desktop application and can operate in two modes: **Simulator** (no hardware required — a built-in synthetic engine model drives every panel, ideal for practice, training, and offline map work) and **Live** (connected to a real vehicle over OBD-II, and to a flash interface for writing ROMs).

For live monitoring, data logging & diagnostics

DORA talks to the vehicle's OBD-II port using the Python obd library, which works with any **ELM327-compatible adapter**:

- A USB ELM327 adapter (appears as a standard COM port — most reliable, no pairing step), or a Bluetooth ELM327 adapter (paired to Windows first, then appears as an outgoing/SPP COM port), or a WiFi ELM327 adapter bridged through a serial-over-IP driver.
- A USB-A to Mini/Micro-USB cable if your adapter isn't a direct dongle.
- The vehicle's OBD-II (SAE J1962) port — a 16-pin trapezoidal connector, almost always within reach of the driver's seat: under the dash, near the steering column, or behind a small trim panel. It is unlocked and unpowered until the ignition is on.
- The correct USB-to-serial driver for your adapter's chipset (commonly CP2102 or FTDI) installed on Windows, so the adapter enumerates as a COM port in Device Manager.

Additionally, for ECU flashing (writing a tune to the car)

Flashing uses a lower-level protocol than OBD-II live data, and DORA supports three transport paths, implemented in `ecu/flash_protocol.py` and `ecu/j2534.py`:

- **KWP2000 over K-line** (ISO 14230 / ISO 9141-2) — a serial K-line interface cable connected via USB, addressed by COM port and baud rate (10400 baud is the KWP2000 standard fast-init rate).
- **UDS over CAN** (ISO 14229-1 / ISO 15765-2) — a USB-to-CAN interface supported by `python-can`, addressed by channel name, TX/RX arbitration IDs, and CAN bitrate (500,000 bps is the common default).
- **A J2534 "pass-thru" device** — the professional standard for ECU flashing (e.g. Tactrix OpenPort 2.0, Drew-Tech Mongoose, and similar). Install the vendor's Windows driver/DLL first — DORA auto-detects any registered J2534 device from the Windows registry, no separate configuration needed.
- A battery charger/maintainer connected to the vehicle for the whole flash procedure — a voltage sag mid-write can corrupt the ECU.
- The tuned ROM (`.bin`) file you intend to write, and ideally a verified-good backup of the original/stock ROM.

Installing DORA

There are two ways to run DORA. Pick whichever matches how it was provided to you.

Option A — Packaged application (recommended for most users)

- 1 Copy the DORA application folder (containing `ProjectDora.exe` and its `_internal` support folder) to your PC.
- 2 Double-click `ProjectDora.exe` to launch — no Python installation is required, everything is bundled.
- 3 On first run, the Setup Wizard opens automatically to walk you through initial configuration (see Section "BUSINESS > Setup Wizard" in Part III).

Option B — Running from source (developers)

- 1 Install Python 3.11 or 3.12 (Windows 10/11 is the primary supported platform, as the UI is built on Tkinter).
- 2 Clone or copy the repository, then open a terminal in `ecu_tune/ecu_tuner`.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

- 1 Copy `.env.example` to `.env` and fill in your configuration (see next section).
- 2 Launch the app:

```
python main.py
```

The app opens straight into the graphical shell (sidebar + dashboard) — there is no separate CLI menu to navigate first.

Configuration (.env)

All runtime settings are read from a `.env` file placed next to the executable (packaged install) or in `ecu_tune/ecu_tuner/` (source install). Never commit or share this file — it can contain API keys.

Variable	Default	Purpose
LICENCE_KEY	(empty)	A DORA subscription licence key (starts DORA-...) from projectdora.com. When set, all AI analysis calls route through the DORA backend — no personal API key needed.
BACKEND_URL	https://api.projectdora.com	Backend endpoint used when LICENCE_KEY is set.
ANTHROPIC_API_KEY	(empty)	Only needed if you are NOT using a LICENCE_KEY (developer/self-hosted mode) — routes AI analysis directly to the Anthropic API.
OBD_PORT	COM3 (Windows)	Default serial port pre-filled in the Connect to Vehicle dialog.
OBD_BAUD_RATE	38400	Default OBD-II baud rate — 9600 / 38400 / 57600 / 115200 are supported.
LOG_LEVEL	INFO	Application log verbosity: TRACE / DEBUG / INFO / WARNING / ERROR / CRITICAL.
LOG_DIR	logs	Folder where data-log CSV files and the flash history log are written.

Which AI setting do I need?

- Most users: set only `LICENCE_KEY` after subscribing — DORA handles the AI connection for you.
- Developers running from source without a subscription: set `ANTHROPIC_API_KEY` instead.
- If neither is set, AI Analysis panels are disabled but every other feature (Map Editor, Live Monitor, ROM tools, Flash, etc.) still works normally.

Connecting DORA to Your Vehicle

- 1 Plug the OBD-II adapter into the vehicle's OBD-II port, and connect it to your PC (USB) or pair it over Bluetooth.
- 2 Turn the vehicle's ignition to **ON** — the engine does not need to be running for the adapter to establish a connection.
- 3 Launch DORA.
- 4 In the app header, click the **connection pill** (reads "Simulator" by default, top right) to open the **Connect to Vehicle** dialog.
- 5 Under **Live OBD-II**, click **Refresh** to list available COM ports, then select the port matching your adapter.
- 6 Set the **Baud** rate — 38400 is the default for most ELM327 adapters; try 9600, 57600, or 115200 if the connection fails.
- 7 Click **Connect**. DORA attempts the handshake in a background thread — this can take a few seconds.
- 8 On success the dialog shows "Connected on COMx" and the header connection pill turns green: "Live OBD-II — COMx". The header also carries a **mode badge** (SIM / LIVE) and a **Safety badge** that reads "Safe" while all monitored values are within limits.
- 9 If the connection fails you'll see an inline error. Double-check the adapter is seated, the port is correct, and the ignition is on — or click **Use Simulator** to keep working offline.
- 10 To end a live session, reopen the connection dialog and click **Disconnect** — DORA falls back to the simulator automatically so panels never go blank.

Flashing uses a separate connection

- The header connection pill only controls the OBD-II live-data link (Live Monitor, logging, fault codes). ECU flashing (ECU > ECU Flash) has its own protocol/port fields inside that panel, since flashing typically needs a different cable/interface than everyday OBD-II monitoring — see Part III.

The App Shell — Sidebar, Header & Shortcuts

Every tool in DORA lives inside one persistent window made of four regions:

- **Header** (top) — the DORA logo (click to collapse/expand the sidebar), a breadcrumb showing the current section, a tear-off button (pops the current panel out into its own window), the notification bell, the connection pill, the Safety badge, and the SIM/LIVE mode badge.
- **Sidebar** (left) — a scrollable, grouped list of every tool, organized into MONITORING, TUNING, AI, ANALYSIS, ROM TOOLS, ECU, HARDWARE, CALC, REPORTS, WORKFLOW, BUSINESS, and SYSTEM sections (these match the chapters in Part III of this manual).
- **Content area** (center) — the active panel.
- **Status bar** (bottom) — current ROM checksum status, last flash timestamp, and a reminder of the Ctrl+K shortcut.

Shortcut	Action
Ctrl + K	Open the command palette — search and jump to any panel by name.
Ctrl + 1	Jump to Live Monitor.
Ctrl + 2	Jump to Map Editor.
Ctrl + 3	Jump to Dyno Test.
Ctrl + 4	Jump to AI Analysis.
Ctrl + 5	Jump to ECU Flash.
Ctrl + 6	Jump to Fault Codes.
Ctrl + 0	Return to Home.
K (in Live Monitor)	Arm trigger-based data logging.
C (in Live Monitor)	Open the Gauge Editor to rearrange the gauge grid.

How the Safety Guard Works

DORA's **SafetyGuard** is a single, centrally-imported module (`safety/safety_guard.py`) that every write path runs through. It cannot be disabled from the UI, and no panel bypasses it — the same check applies whether a change comes from you dragging a cell in the Map Editor, a manual adjustment, or an AI recommendation.

- **validate_map_change()** checks every proposed ROM write against a maximum per-cell delta and a maximum total number of cells changed in one session — large or sweeping changes are rejected outright rather than silently clipped.
- **validate_live_data()** continuously checks live sensor values (boost, coolant temp, EGT, knock) against hard limits during monitoring and dyno runs — this is what drives the header's Safety badge and any on-screen alarms.
- Every ROM write creates a `.bak` backup copy before overwriting the target file (see ROM History in Part III).
- AI-proposed adjustments are never applied automatically — the AI can only recommend, and every recommendation still passes through the same per-cell/total-cell scrutiny as a manual write before a human can confirm it.

Hard Limits Reference

All limits are defined once in `safety/limits.py` and imported everywhere else — nothing hardcodes a different number. These are the shipped defaults:

Parameter	Limit	Why
Minimum WOT AFR	11.5 : 1	Leaner risks detonation / piston damage ($\approx$ lambda 0.78).
Maximum idle AFR	15.5 : 1	Stoich (14.7) plus a lean-idle tolerance.
Max fuel trim (ST/LT)	$\pm 25\%$	Beyond this the base fuel map itself needs correcting.
Max ignition advance	35° BTDC	Typical safe ceiling for most modified engines.
Max idle ignition advance	20° BTDC	Idle timing rarely needs more.
Max global ignition shift / session	$\pm 3^\circ$	Caps how far a single session can move the whole map.
Max boost	1.8 bar (≈ 26 psi)	Absolute ceiling for most modified engines.
Boost fuel-cut limit	2.0 bar	Hard ECU-side ceiling above the boost target.
Max coolant temperature	105°C	Above this, risk of head gasket failure.
Max EGT	900°C	Above this, risk of turbo / valve damage.
Max knock intensity	0.75 (normalised 0–1)	Beyond this, timing must retard.

Knock retard threshold range	0.5° – 5.0°	Valid range for the configurable retard-per-knock-event setting.
Lambda range at WOT	0.75 – 1.05	Below 0.75 = dangerously rich (fouling); above 1.05 = too lean.
Max single-cell map change	±5% per session	Forces incremental, verifiable tuning steps.
Max cells changed per write	20 cells	Limits blast radius of any one write.
Hard rev limit ceiling	8,500 RPM	Absolute ceiling regardless of engine.
Idle RPM range	500 – 1,200 RPM	Valid target range for idle speed control.
Max intake cam advance	40°	VVT/cam control ceiling.
Max exhaust cam retard	20°	VVT/cam control ceiling.
Max IAT fuel correction	5% per 10°C >25°C	Intake-air-temperature compensation ceiling.

If a change is blocked

- A rejected change is never partially applied — split it into smaller, session-by-session steps instead of trying to raise the limit.
- Limits are conservative defaults intended to be safe across most modified engines, not a guarantee of safety for every build — always cross-check against your specific engine's known tolerances.

Part III

Complete Feature Reference

Organized to match the sidebar, group by group. Every panel reachable from the sidebar is documented — what it's for, every button and field, how to use it step by step, and any technical detail worth knowing.

MONITORING

Live Monitor

Sidebar: MONITORING > Live Monitor (Ctrl+1) — opens as its own window

The main real-time multi-gauge dashboard: streams RPM, boost, throttle, O2 voltage, coolant temp, MAF, and short/long-term fuel trim as scrolling strip charts, with click-to-edit map access, alarms, an oscilloscope, GPS overlay, and trigger-based logging all built into the same window.

KEY UI ELEMENTS

- 3×3 grid of strip-chart graphs plus a text-only live-values panel. Editable channels (Boost, Throttle, MAF, STFT, LTFT) are bordered purple with a pencil glyph; read-only channels show a [R] lock glyph.
- Live numeric overlay on every chart, timestamp readout top-right.
- Keybinds: P pause/resume, B/T/M/S/L open the map editor for Boost/Throttle/MAF/STFT/LTFT, O oscilloscope, G GPS track overlay, A threshold alarms, K trigger-based logging, C gauge layout editor.

HOW TO USE IT

- 1 Open MONITORING > Live Monitor (or Ctrl+1) — data streams immediately at 100ms/frame.
- 2 Click any purple-bordered graph, or press its hotkey, to open the built-in Cell Editor for that channel's RPM×Load map: click a cell, type a value, press Enter to commit (R resets the cell, A resets the whole map, Ctrl+Z undoes).
- 3 Press S or click Save & Close to commit — the change is validated by the Safety Guard before a confirmation dialog applies it.
- 4 Press A to set min/max alarm thresholds per channel; a breached channel flashes its border red and beeps once per new breach.
- 5 Press K to arm Trigger-Based Logging: choose a start condition (e.g. RPM > 3000) and stop condition (e.g. RPM < 1000); Save Log CSV exports the captured rows once it fires.
- 6 Press O for the Oscilloscope (toggle channels, adjust Zoom, click to drop cursor lines), or G for a synthetic GPS Track Overlay colour-coded by a selected channel.
- 7 Press C to open the Gauge Layout Editor for a customizable gauge grid (see its own entry below).

TECHNICAL NOTES

- Data source is `ecu.snapshot()` — works identically against the simulator and real hardware; MAF is used as the "load" axis for every RPM×Load map lookup.
- Every map save here is checked by `SafetyGuard.validate_map_change()` against the same global limits as everywhere else (max 5% delta/cell, max 20 cells/session) — violations block the write entirely, no partial writes.
- Hard reference lines are drawn on-chart at Coolant >105°C and Boost >1.8 bar.
- If a ROM is loaded, saved changes write through to `output_<Channel>.bin`; otherwise they're appended to a timestamped CSV under `logs/`.

Gauge Layout Editor

Access: press C from Live Monitor — not a sidebar entry

A drag-and-drop gauge grid designer — assign a parameter, visual style, range, and colour to each cell of a 2×2, 3×3, or 4×4 gauge grid, then save/load the layout as JSON.

KEY UI ELEMENTS

- Toolbar: Save Layout, Load Layout, Reset, grid-size radios (2×2 / 3×3 / 4×4).

- Left: the live gauge grid. Right: Cell Properties — Parameter dropdown (RPM, Boost, Throttle, Coolant, IAT, MAF, Lambda, AFR, Fuel Trim ST/LT, Timing Advance, Battery, Oil Pressure, Vehicle Speed), Style radios (Bar / Arc / Digital / Needle), Min/Max/Unit fields, colour swatch, and a Preview Value slider.

HOW TO USE IT

- 1 Click a cell to select it (teal highlight).
- 2 Choose a Parameter (auto-fills Min/Max/Unit, editable), a Style, and a colour.
- 3 Drag the Preview Value slider to see the gauge render live.
- 4 Drag one cell onto another to swap configurations; right-click for Copy/Paste/Clear Cell.
- 5 Click Save Layout to export JSON, or Load Layout to restore a saved one.

TECHNICAL NOTES

- This is a design tool over an in-memory cell list — it does not read live ECU data itself (the Preview slider fakes values), and layouts are consumed via a callback rather than wired to a specific dashboard by default.

Knock Logger

Sidebar: *MONITORING > Knock Logger*

Real-time knock-event capture that bins every detected knock-retard event into an RPM×Load heat map, alongside a scrolling log of individual events.

KEY UI ELEMENTS

- Start Logging / Stop / Clear / Export CSV buttons.
- 14×10 (RPM×Load) heat-map canvas with a Low→High colour legend.
- Right-hand Knock Events scrolling log (red text for events $\geq 2.0^\circ$, amber for lighter events).

HOW TO USE IT

- 1 Click Start Logging — polls the ECU every 0.25s.
- 2 Any sample with knock retard $> 0.1^\circ$ brightens its RPM/Load cell (colour = the max of previous and new value) and logs a timestamped event.
- 3 Click Stop to halt polling.
- 4 Click Export CSV to save all events (time, rpm, load, knock).

TECHNICAL NOTES

- RPM bins span 800–7000 (14 steps), Load bins 20–200 (10 steps); cell colour uses a 7-stop gradient normalised against a 5.0° ceiling.

Log Viewer

Sidebar: *MONITORING > Log Viewer*

Spreadsheet-style playback tool for previously captured CSV logs, with a channel selector, timeline scrubber, and playback controls.

KEY UI ELEMENTS

- Load Log... button; left-hand Channels checklist with a drag handle on each row for column reordering.
- Timeline scrubber with a live time readout; Play / Stop and a Speed slider (0.25×–8×).
- Stats strip (min/max/avg for the first 6 numeric columns); scrollable data table; Export CSV.

HOW TO USE IT

- 1 Click Load Log... and choose a CSV.
- 2 Toggle channel checkboxes to show/hide columns; drag a row's handle to reorder.
- 3 Drag the Timeline scrubber, or click Play, to step through the log.
- 4 Right-click a row for Copy Row / Jump To In Timeline / Export Visible Columns...
- 5 Click Export CSV to save only the currently-checked columns.

TECHNICAL NOTES

- Requires a time_s column for the scrubber/stats range to activate.
- The table renders in 500-row chunks around the scrub position for performance on large logs.

Fuel Trims

Sidebar: MONITORING > Fuel Trims

Live-sampled short/long-term fuel-trim heat maps per RPM×Load cell, alongside AFR estimated from O2 voltage, for spotting fueling problems across the operating range.

KEY UI ELEMENTS

- Start Sampling / Stop / Reset buttons; view-mode radios (STFT / LTFT / Combined).
- Two side-by-side 14×10 heat-map canvases (Short Term / Long Term Fuel Trim %).
- Live readout strip: RPM, Load, STFT, LTFT, AFR, O2V.

HOW TO USE IT

- 1 Click Start Sampling — polls the ECU every 0.5s.
- 2 Each sample updates a running average for its nearest RPM/Load bin; red = lean (>+3%), blue = rich (<-3%), dark green = near zero.
- 3 Watch the readout strip: STFT/LTFT turn amber past ±5%, red past ±10%; AFR turns red outside 13.5–15.2.
- 4 Click Reset to clear accumulated averages, or Stop to halt polling.

TECHNICAL NOTES

- AFR is approximated from O2 voltage: $AFR \approx 14.7 \times (1 - (o2v - 0.45) / 0.45)$.
- Display/diagnostic only — this panel does not write corrections back to the ECU (see Closed Loop in the TUNING chapter for that).

Dyno Test

Sidebar: MONITORING > Dyno Test (Ctrl+3) — opens as its own window

A full virtual/live dynamometer: runs simulated, live, steady-state, or scripted-sequence pulls, renders a multi-axis HP/Torque/Boost/Lambda/Timing chart with run history and comparisons, applies SAE/ISO/DIN atmospheric correction, and produces PDF/CSV reports.

KEY UI ELEMENTS

- Top toolbar: Mode radios (Simulated / Live / Steady-state / Sequence), Run/Stop/Clear, Auto-detect WOT checkbox.
- Chart tab: 4-row stacked plot (Power/Torque, Boost, Lambda, Timing) with Smooth slider, Target-curve overlay, before/after comparison window, and a Run History sidebar for selecting 2 runs to delta-compare.
- Live Gauges tab: 12 threshold-coloured bar gauges.
- Correction tab: Temperature/Pressure/Humidity, Standard dropdown (SAE/DIN/ISO), Driveline Loss %.

- Load Control tab: full sweep / steady-state RPM lock / custom sequence builder, PID tuning for the simulated brake.
- Math Channels tab: user-defined formula channels evaluated per data point.
- Session tab: car/report info, Export PDF/CSV/PNG, Save/Load Session (JSON), Import/Export DataLog CSV, per-run lock & notes, and an AI button per run.

HOW TO USE IT

- 1 Choose a Mode radio, click Run, and name the run when prompted.
- 2 Simulated runs a full virtual sweep instantly (using ROM maps if loaded); Live samples an actual WOT pull via the connected ECU (Auto-detect WOT arms automatically past 65% throttle); Steady-state holds a target RPM via PID; Sequence steps through a scripted RPM/hold-time list.
- 3 Click Stop at any time to abort.
- 4 Select two runs' circle markers in the sidebar to overlay a green/red delta-shaded comparison with a peak-gain callout, or click Before/After for a dedicated comparison window.
- 5 Configure atmospheric correction on the Correction tab and click Apply & Redraw.
- 6 On Session, fill in car info and Export PDF Report, or Save Session to JSON for later reload.
- 7 Click AI next to any run to send a structured summary to the AI tuning agent; if a ROM is loaded, Apply Adjustments writes suggested per-cell deltas straight into ROM maps — gated by the Safety Guard, with automatic revert-all on any violation.

TECHNICAL NOTES

- Live/steady-state/sequence runs pass through SafetyGuard continuously; a breach triggers a SAFETY ABORT dialog listing every violated limit and stops the runner.
- A run is auto-annotated with a dotted marker wherever knock >0.6 and RPM >5000 occur together.
- Data-log import/export requires pandas; missing-dependency errors show explicitly rather than failing silently.

TUNING

Map Editor

Sidebar: TUNING > Map Editor (Ctrl+2)

DORA's flagship ECU-map editing tool — a full-screen, keyboard-driven RPM×Load grid editor with a live crosshair tied to the current engine operating point, plus 3D surface views, smoothing, region operations, interpolation, multi-tune comparison, axis rescaling, search, bookmarks, and CSV/Excel import/export. This is DORA's closest analogue to WinOLS/ECUFlash/TunerPro's table editor.

KEY UI ELEMENTS

- Launcher screen (TUNING > Map Editor): channel radio buttons (Boost, Throttle, MAF, Short/Long Term FT), optional ROM file field, Open Map Editor button — this spawns the full editor as a separate process.
- Editor window: scrollable RPM×Load grid, a 270px right-hand tool panel, and a bottom stats bar (Modified: N cells | Min | Max | Mean).
- Right panel: cell info readout, keyboard cheat-sheet, colour legend, value scale gradient, and ten feature buttons — Bookmarks, Map Search, Export CSV/XLSX, Import CSV/XLSX, Axis Rescale, Region Ops, Interpolate, Compare Tunes, 3D Compare, 3D Surface View — plus Save & Close.
- Keybinds: 0–9 . – type a value, Enter commit, Escape cancel, arrow keys move selection, R/A reset cell/reset all, S save & close, Ctrl+Z undo (up to 50 steps); Shift+click twice to define a rectangular region for the bulk tools.

HOW TO USE IT

- 1 From TUNING > Map Editor, pick a channel (and optionally a ROM file for on-disk write context), then click Open Map Editor.
- 2 Click a cell, type a value, press Enter to commit — values outside the channel's safety limits are rejected inline with a red BLOCKED flash.
- 3 Shift+click two opposite corners to define a region, then click Region Ops for Scale by %, Add constant, or a linear ramp across the load axis.
- 4 Click Interpolate for Smooth (Gaussian/box blur), Fill Empty Cells (bilinear), Blend Corners, or Extrapolate — each scoped to the current region or the whole map.
- 5 Click Compare Tunes to load another saved tune and see a colour-coded delta grid plus summary stats, or 3D Compare / 3D Surface View for a rendered surface.
- 6 Click Axis Rescale to non-destructively re-space RPM/Load breakpoints (re-interpolates existing data onto the new grid).
- 7 Use Map Search to find cells in a value range; use Bookmarks to name and jump back to specific cells.
- 8 Export CSV/XLSX to save the grid, or Import CSV/XLSX to load one back in (values are clipped to the channel's min/max on import).
- 9 Press S or click Save & Close to write the result and close the editor.

TECHNICAL NOTES

- Live cell crosshair: while Live Monitor or a Dyno run is active, it writes the current RPM/load operating point to a sidecar file every ~250ms; the Map Editor polls it and highlights the nearest cell in cyan with a fading teal breadcrumb trail of recent history — this works purely through the filesystem, with no direct process link.
- The editor runs as an independent OS subprocess (not embedded) because its own matplotlib/Tk event loop would otherwise conflict with Live Monitor's animation loop.

- Per-cell entry is hard-blocked by channel-specific safety limits (Boost 0–1.8 bar; Fuel Trim $\pm 25\%$) as you type; other channels only clamp to their editable range. The full `SafetyGuard.validate_map_change()` session-level check (max 5% delta/cell, max 20 cells/session) runs back in the parent process when a change originates from Live Monitor's own in-process Cell Editor — the standalone Map Editor subprocess writes its result directly to disk without a second pass beyond the per-cell limits already enforced while typing.
- Four supporting tools exist in the codebase — a categorised Map Browser, a two-tab Axis Editor (independent per-axis label/unit/scaling metadata), a per-map Annotation Panel (notes/tags), and a Color Scheme Editor (custom heat-map gradients) — but none are currently wired into the running Map Editor, so they aren't reachable from the app today.

Manual Tune

Sidebar: TUNING > Manual Tune

A single scrollable form covering every safety-checked tuning parameter in the app — fuel, AFR, ignition, boost, idle, temperature, rev limiter, VVT — with range validation against the Safety Guard's limits before any value reaches the ECU.

KEY UI ELEMENTS

- Eight sections: Fuel, Air/Fuel Ratio, Ignition, Boost, Idle, Temperature, Rev Limiter, VVT/Cam Timing — each field has hint text and many carry a hover tooltip.
- Quick-preset buttons: stage-specific presets (Stage 1/2/3, shown when opened from a matching job service type), Performance Tune, Fuel Trim Test.
- Buttons: Apply Adjustments (or Apply + Run AI in AI-chained mode), Clear All, Reset to Defaults, Save Preset, Load Preset.

HOW TO USE IT

- 1 The form auto-prefills from simulator defaults and any active ECU overrides on open.
- 2 Edit any field — out-of-range values highlight dark red on focus-out.
- 3 Optionally click a stage preset (Stage 1: 0.8 bar boost / 25° WOT timing; Stage 2: 1.2 bar / 28°; Stage 3: 1.5 bar / 32° / 8000 RPM rev limit) or Performance Tune / Fuel Trim Test to bulk-fill demo values.
- 4 Click Apply Adjustments — every non-empty field is range-checked against `SafetyGuard.LIMITS`; any failure blocks the entire submission with an error list.
- 5 Confirm the summary dialog listing every value about to be written.
- 6 If opened in Apply + Run AI mode, a 60-second log-and-analyse pass runs automatically afterward, following the same safety-checked apply flow as AI Analysis.
- 7 Use Save Preset / Load Preset to persist or restore the whole form as JSON.

TECHNICAL NOTES

- Validation is centrally defined against `SafetyGuard.LIMITS`, so this form cannot bypass the same ceiling used everywhere else in the app (max fuel trim %, min AFR at WOT, max ignition advance, max boost bar, max coolant enrichment temp, and more — see Part II).

Closed Loop

Sidebar: TUNING > Closed Loop

Wideband-lambda auto-tune assistant — samples live lambda error against configurable cruise/WOT targets and accumulates a correction log that can be pushed to the fuel map.

KEY UI ELEMENTS

- Wideband source radios (Simulator O2 is the only functional option); Target lambda (cruise/WOT), Max correction per cell (%/pass), Min RPM for sampling fields.
- Smooth corrections checkbox; "Apply corrections live to ECU" checkbox.
- Start Auto-Tune / Stop / Apply Fuel Map Corrections / Clear buttons.
- Left: live gauge strip (Lambda, AFR, RPM, Load, Correction %). Right: colour-tagged Correction Log.

HOW TO USE IT

- 1 Set target lambdas and max per-pass correction, click Start Auto-Tune.
- 2 Every 0.5s the panel computes lambda from O2 voltage, compares to the cruise or WOT target (auto-switches past 80% throttle), and logs a correction clamped to $\pm$ Max correction.
- 3 If "Apply corrections live" is checked, each sample also nudges the simulator's fuel delta in real time.
- 4 Click Stop once enough samples are collected, then Apply Fuel Map Corrections to average the log and push it to the simulator.
- 5 Click Clear to wipe the correction log.

TECHNICAL NOTES

- This panel writes only a global fuel delta override to the simulator — it does not perform true per-cell ROM writes and does not invoke the Safety Guard's map-change check, since it never touches a map array directly.

Launch Control

Sidebar: *TUNING > Launch Control*

Configuration for three motorsport strategies — 2-step launch control, full-throttle flat-shift, and anti-lag — with simulator apply and JSON export.

KEY UI ELEMENTS

- Launch Control: Launch RPM limit, Ignition retard, Fuel enrichment, hold-on-brake+clutch field, Cut strategy radios (Ignition/Fuel/Both).
- Flat Shift: Ignition cut on clutch press (ms), Timing retard during shift, Anti-surge boost hold.
- Anti-Lag (ALS): ignition retard, idle air addition, max EGT limit, with a red "track/competition use only" warning.
- Apply to ECU (Simulator) and Export Config buttons.

HOW TO USE IT

- 1 Tick the Enable checkbox for whichever strategy is needed.
- 2 Fill in the numeric fields for that strategy.
- 3 Click Apply to ECU (Simulator) — only the Launch Control RPM limit and ignition retard are currently pushed as live overrides.
- 4 Click Export Config to save all three sections to one JSON file.

TECHNICAL NOTES

- Flat-Shift and Anti-Lag fields are captured and exported but not yet wired to live simulator behaviour beyond the export file.

Flex Fuel

Sidebar: *TUNING > Flex Fuel*

Ethanol-content sensor emulator with linear E0↔E100 blending of fuel trim, ignition advance, and boost allowance.

KEY UI ELEMENTS

- Large live "Ethanol Content" readout with category label (E0 Pure Gasoline / Low / Mid / High / E100).
- Simulate ethanol % slider (0–100); E0/E100 fuel-trim fields, E85 ignition-advance field, E85 boost-allowance field.
- Calculated Blend box (live-interpolated fuel trim / ignition / boost); Apply Blend to ECU (Simulator).

HOW TO USE IT

- 1 Set the E0/E100 fuel trim, E85 ignition advance, and E85 boost allowance base values.
- 2 Drag the Simulate ethanol % slider — the Calculated Blend values update instantly.
- 3 Click Apply Blend to ECU (Simulator) to push the blended values as overrides.

TECHNICAL NOTES

- All three outputs are simple linear interpolations between the E0 and E100 endpoints. No Safety Guard check applies since this writes scalar overrides, not map cells.

Boost Control

Sidebar: *TUNING > Boost Control*

Three-tab boost management: target boost table, wastegate duty-cycle table, and overboost protection settings.

KEY UI ELEMENTS

- Boost Target (bar): 10×5 RPM×Load editable grid.
- Wastegate DC (%): matching 10×5 grid.
- Overboost Protection: enable checkbox, cut threshold (default 1.8 bar), cut duration, ignition retard before cut, fuel-cut checkbox, spike-detection window/threshold.

HOW TO USE IT

- 1 Edit boost-target cells directly, then click Apply to ECU — this pushes the 3500 RPM @ 100% load cell as the simulator's boost target override.
- 2 Edit the wastegate duty-cycle table on its tab (Apply here is currently confirmation-only, no simulator write).
- 3 Set overboost protection thresholds and click Apply Protection Settings (informational only).
- 4 Use Export JSON on any tab to save that table.

TECHNICAL NOTES

- Default boost-target table ramps 0→1.6 bar peak around 3000–4000 RPM at high load; default wastegate table ramps 0→85%.

Idle Control

Sidebar: *TUNING > Idle Control*

Idle-air-control strategy plus three coolant-temperature-indexed tables: idle RPM target, idle ignition trim, and cold-start fuel enrichment.

KEY UI ELEMENTS

- IAC control mode radios (Open-loop / Closed-loop PID / PWM stepper); PID Kp/Ki/Kd, throttle-close dashpot, IAC min/max position.
- Three 10-column tables keyed to coolant temps –20°C–100°C.
- Apply to ECU / Export JSON.

HOW TO USE IT

- 1 Choose an IAC control mode and tune PID gains if closed-loop.

- 2 Edit the three coolant-indexed rows directly.
- 3 Click Apply to ECU — pushes the 80°C (warmed-up) idle RPM value as the simulator override.
- 4 Click Export JSON to save all three tables plus PID/dashpot/IAC settings.

TECHNICAL NOTES

- Default idle RPM ramps from 1400 RPM cold (–20°C) down to 800 RPM fully warm.

Map Switching

Sidebar: *TUNING > Map Switching*

Defines up to six named tuning stages (fuel trim / ignition advance / rev limit / boost target) and an optional automatic input-triggered switch between them.

KEY UI ELEMENTS

- Four quick-select stage buttons plus an Active: <stage> label.
- One card per stage (Stage 1 Economy, Stage 2 Sport, Stage 3 Race, Stage 4 Track/Unlimited, Custom A, Custom B) with 4 parameter fields each.
- Conditional Switching: enable checkbox, Trigger source dropdown (Button / Launch mode active / Coolant temp threshold / Time-of-day / GPS speed), Switch-to-stage dropdown.
- Apply Stage Now / Export All Stages.

HOW TO USE IT

- 1 Edit each stage card's four parameters.
- 2 Click a stage button to mark it Active.
- 3 Click Apply Stage Now — pushes the active stage's values as simulator overrides.
- 4 Optionally enable Conditional Switching and choose a trigger/target stage (configuration only — not yet wired to a live trigger evaluator).
- 5 Click Export All Stages to save every stage to JSON.

VVT/Cam Control

Sidebar: *TUNING > VVT/Cam Control*

Variable-valve-timing cam-angle target table editor supporting several real-world VVT systems.

KEY UI ELEMENTS

- VVT System Type dropdown (Generic OBD, Honda VTEC, Toyota VVT-i, BMW VANOS single/double, Ford Ti-VCT, Subaru AVCS, Nissan VVEL); VVT enabled checkbox.
- VTEC engagement RPM field (appears only for VTEC systems).
- 11×7 RPM×Load Cam Angle Target (°) table (positive=advance, negative=retard).
- VVT Limits: max advance, max retard, hold-position-below-coolant-temp.
- Apply VVT Map / Export JSON.

HOW TO USE IT

- 1 Select the engine's VVT system type.
- 2 Edit the cam-angle table (defaults ramp 0° at idle up to ~40° around 3500–5000 RPM at high load).
- 3 Set the VVT limit fields.

- 4 Click Apply VVT Map (confirmation dialog only — no live simulator write) or Export JSON.

Overrun Fuel Cut

Sidebar: TUNING > Overrun Fuel Cut

Deceleration fuel-cut-off (DFCO) configuration, plus an optional cosmetic "exhaust pop" overrun ignition-advance feature.

KEY UI ELEMENTS

- Enable DFCO checkbox; engage/re-engage RPM, throttle-closure threshold, min coolant temp, re-engagement delay fields; smooth re-engagement checkbox.
- 10×5 RPM×Coolant DFCO permission map (1=cut allowed, 0=no cut).
- Overrun Ignition Advance: enable checkbox (cosmetic pops/crackles), overrun retard, overrun fuel re-inject %.
- Apply DFCO Config / Export JSON.

HOW TO USE IT

- 1 Enable DFCO and set the engage/re-engage thresholds and coolant gate.
- 2 Edit the RPM×Coolant permission grid if needed.
- 3 Optionally enable the cosmetic overrun-pop feature and set its parameters.
- 4 Click Apply DFCO Config — pushes only the boolean `dfco_enabled` flag as a simulator override; Export JSON saves the full configuration including the permission grid.

AI

AI Analysis

Sidebar: AI > AI Analysis (Ctrl+4)

Runs a single guided AI tuning pass: optionally identify a loaded ROM, log 60 seconds of engine data, and send a statistical summary to the AI tuning agent for map-cell adjustment proposals.

KEY UI ELEMENTS

- ROM file (optional) field with Browse and drag-and-drop.
- Large scrolling output/log area.
- Run AI Analysis button, plus shared Live Dashboard / Dyno Test buttons.

HOW TO USE IT

- 1 Optionally browse to or drop a ROM binary.
- 2 Click Run AI Analysis.
- 3 If a ROM was supplied, DORA identifies it and offers to diff it against the closest stock match in the database.
- 4 The app logs 60 seconds of engine data (10Hz) with a live countdown, then summarises it statistically.
- 5 The summary (plus any ROM context) is sent to the AI tuning agent, which prints a summary, confidence, warnings, and a numbered list of proposed map-cell adjustments (map[row,col] delta% — reason).
- 6 Confirm Apply to run every adjustment through the Safety Guard; violations block the write and print in red, otherwise the overrides are applied to the ECU.

TECHNICAL NOTES

- This window always safety-checks before writing to the ECU — it is a thread-backed wrapper around the same TuningAgent/SafetyGuard pipeline used elsewhere, not the newer Anthropic-SDK panel described in AI Analysis+ below.

AI Analysis+

Sidebar: AI > AI Analysis+

A dedicated 6-tab AI workspace that calls the Anthropic API directly for live tuning suggestions, auto-tune correction analysis, anomaly detection, knock-risk mapping, full map-optimisation reports, and known-good safety comparisons.

KEY UI ELEMENTS

- Real-Time Monitor tab: Start/Stop polling every 5s into a colour-tagged live table; auto-triggers an AI commentary call roughly every 60s, or Analyze Now forces one.
- Auto-Tune Mode tab: load a datalog CSV, Compute Corrections bins it into a 14×10 RPM×Load grid and computes per-cell fuel-trim correction (clamped ±5%) as a green/red heatmap; Generate AI Report sends the table to Claude for a summary and safety notes; Export Corrections CSV.
- Anomaly Detection tab: loads any CSV, auto-detects channels, runs a rolling z-score check ($\sigma > 2.5$ = Warning, $\sigma > 3.0$ = Critical) into a sortable results table.
- Knock Risk tab: live scoring or CSV load, bins knock-retard into the same RPM×Load grid as a risk heatmap (green<5% Safe, yellow 5–15% Moderate, orange 15–30% High, red>30% Critical).
- Map Report tab: load a map CSV (+ optional log CSV) and free-text context, Generate Report sends statistics to Claude for a structured report (Executive Summary, Critical Issues, Enrichment/Timing Recommendations, Safety

Notes).

- Known-Good Compare tab: load your map and a reference map (browse or from a searchable known-good library), Compare renders a deviation heatmap, AI Safety Review sends the top-20 highest-deviation cells to Claude for a safety flag review.

HOW TO USE IT

- 1 Pick the tab matching your task.
- 2 Load whatever data that tab needs (live poll, CSV, or map file).
- 3 Click the tab's primary action button (Compute Corrections / Generate AI Report / Start Live Scoring / Generate Report / Compare, etc.).
- 4 Review the rendered heatmap, table, or AI-written report; export to CSV/txt where offered.

TECHNICAL NOTES

- AI calls use the Anthropic Messages API directly (model claude-sonnet-4-6, max 1024 tokens), instructing Claude to act as "an expert ECU calibration engineer" or "safety reviewer" with a capped reply length per prompt.
- If the anthropic package isn't installed or ANTHROPIC_API_KEY isn't set, an orange warning banner shows at the top and AI calls return a warning string instead of failing.
- All AI calls run on a background thread so the UI never blocks while waiting on a response.

ANALYSIS

Fill From Log

Sidebar: ANALYSIS > Fill From Log

Loads a wideband datalog CSV and computes per-cell fuel-map correction percentages by comparing logged AFR against a target AFR, binned by RPM and load.

KEY UI ELEMENTS

- Load CSV button; column-mapping row (RPM/Load/AFR columns, auto-detected from headers, editable).
- Target AFR (default 14.7), Max correction % (default 10.0), Min samples/cell (default 3).
- Calculate Corrections / Apply to Fuel Map / Clear buttons.
- 14×10 correction table (RPM 500–7000, Load 10%–100%) with green (enrich) / red (lean) shading.

HOW TO USE IT

- 1 Load a datalog CSV — column names auto-detect from common headers.
- 2 Correct the column mapping if needed.
- 3 Set Target AFR, Max correction %, and Min samples/cell.
- 4 Click Calculate Corrections — each row bins into an RPM×Load cell; $\text{error\%} = (\text{target} - \text{ afr}) / \text{target} \times 100$, averaged and clamped.
- 5 Review the colour-coded table, then click Apply to Fuel Map to push the average correction to the ECU.

TECHNICAL NOTES

- Apply to Fuel Map currently pushes one global average delta rather than writing each cell individually.

Dyno Overlay

Sidebar: ANALYSIS > Dyno Overlay

Overlays up to 5 dyno pulls (HP and torque vs. RPM) on one chart for direct before/after or run-to-run comparison.

KEY UI ELEMENTS

- Add Run (CSV) / Add Simulator Run / Clear All / Save Chart PNG buttons.
- Coloured chip strip listing loaded runs.
- Two-panel chart: Power on top, Torque below, shared RPM axis.

HOW TO USE IT

- 1 Click Add Run (CSV) and select a dyno CSV (auto-detects RPM/power/torque columns), or Add Simulator Run for a synthetic demo curve — up to 5 runs total.
- 2 Each run gets a fixed palette colour and plots on both axes with a legend.
- 3 Click Save Chart PNG to export at 150dpi (PNG or PDF).

Log Compare

Sidebar: ANALYSIS > Log Compare

Overlays two datalogs (e.g. stock vs. tuned) on a shared sample-index timeline for a chosen channel, with summary statistics.

KEY UI ELEMENTS

- Baseline (A) and Tuned (B) file rows with colour swatches.

- Channel dropdown (rpm, afr, map_kpa, throttle_pos, coolant_temp, ignition_advance, boost_bar, fuel_pw_ms).
- Plot Comparison / Add Sim Data buttons; a 6-value stats strip (A/B min/max/avg).

HOW TO USE IT

- 1 Browse to load Baseline (A) and Tuned (B) logs, or Add Sim Data for 400 synthetic rows of each.
- 2 Pick a channel — the chart re-plots automatically.
- 3 Click Plot Comparison to overlay both logs by sample index.
- 4 Review min/max/avg for both logs in the stats strip.

ROM TOOLS

Several standalone Toplevel implementations exist in the source tree alongside the panels below (a separate ROM Library, Patch Manager, and ROM Diff viewer) but are never imported anywhere else in the app and are not reachable from the sidebar — the panels documented here are the ones you'll actually reach.

ROM Identify

Sidebar: ROM TOOLS > ROM Identify

Identifies an unknown ROM binary against the bundled reference database, optionally diffs it against the closest stock match, and can chain straight into an AI tuning pass using the identified ROM as context.

KEY UI ELEMENTS

- ROM file entry with drag-and-drop + Browse; scrolling output log.
- Identify ROM button, plus shared Live Dashboard / Dyno Test buttons.

HOW TO USE IT

- 1 Browse to or drop a ROM binary.
- 2 Click Identify ROM — prints the best-match make/model/ECU type and confidence to the log.
- 3 Confirm "Diff against best stock match?" to merge a byte-diff against the top match into the ROM context.
- 4 Confirm "Run AI analysis with this ROM context?" to chain into the same 60-second-log → AI-recommendation → safety-checked-apply flow as AI Analysis.

Hex Editor

Sidebar: ROM TOOLS > Hex Editor

Byte-level inspection and editing of a loaded ROM with live multi-format decoding, bad-block detection, pattern search, and several obfuscation/encryption cipher transforms — DORA's equivalent to WinOLS/ECUFlash's hex view.

KEY UI ELEMENTS

- Header: Load ROM..., Save ROM, Export Sel.
- Toolbar: hex search + Find/Next, Goto offset, byte-order radio (BE/LE), "View as" radio (u8/u16/u32/f32), Bad blocks / Show edits overlay checkboxes, Cipher dropdown (XOR, ADD, SUB, ROL, ROR, AES-CBC, AES-ECB) + hex Key field + Apply.
- Main grid: monospace hex/ASCII, 16 bytes/row, click-to-select, arrow-key navigation.
- Right sidebar: Selection panel (decodes the selected byte as u8/u16/u32/f32 in both byte orders simultaneously), Bad Blocks summary, inline Edit Byte field, colour legend.

HOW TO USE IT

- 1 Click Load ROM... — the panel scans for bad blocks (runs ≥ 64 bytes of 0x00 or 0xFF) immediately.
- 2 Click any byte to select it and see all numeric interpretations at that offset.
- 3 Use hex search (space-separated bytes, e.g. "FF 80 00") with Find/Next, or Goto to jump to a decimal or 0x-hex offset.
- 4 To edit: select a byte, type a 2-digit hex value into Edit Byte, press Apply/Enter — the byte turns green (Modified).

- 5 To apply a cipher: enter a hex key, pick the cipher, click Apply (confirmed with a destructive-action warning).
- 6 Click Save ROM to write the buffer to a new file, or Export Sel to dump 256 bytes from the current selection.

TECHNICAL NOTES

- XOR: byte XOR key[i mod len] (repeating-key XOR, common on older ECU obfuscation schemes). ADD/SUB: additive byte obfuscation, reversible by re-running the inverse. ROL/ROR: bitwise rotate by key[0] mod 8, applied uniformly to every byte.
- AES-ECB/AES-CBC use the optional cryptography package; the key is padded/truncated to a valid AES length, the buffer zero-padded to a 16-byte boundary, and CBC assumes a zero IV (ROMs rarely store one inline). All ciphers are symmetric-toggle — re-applying the same cipher/key decrypts what was just encrypted, useful for probing whether a region is genuinely enciphered.
- Bad-block detection is a heuristic for erased flash/padding, not necessarily invalid data.

Checksum Mgr

Sidebar: ROM TOOLS > Checksum Mgr

Computes and verifies multiple checksum/CRC algorithms over a loaded ROM, auto-detects where a stored checksum lives near the end of the file, and can patch a corrected CRC32 into the last 4 bytes.

KEY UI ELEMENTS

- ROM path + Browse; algorithm checkbox grid (Sum8, Sum16, Sum32, XOR8, CRC8, CRC16, CRC32 — all on by default).
- Verify All / Auto-Detect Location / Patch ROM / Clear buttons; scrolling coloured output log.

HOW TO USE IT

- 1 Browse to a ROM — size and an MD5 fingerprint print immediately.
- 2 Click Verify All to compute every checked algorithm over the whole file.
- 3 Click Auto-Detect Location to scan the last 256 bytes for a stored value matching a freshly computed CRC16/CRC32 (checked both byte orders).
- 4 Click Patch ROM to recompute CRC32 over all-but-the-last-4-bytes and write it big-endian into the final 4 bytes of a new file.

TECHNICAL NOTES

- Implements Sum8/16/32, running-XOR8, custom CRC8 (poly 0x07), CRC16 (poly 0xA001, Modbus-style), and CRC32 (via binascii).

ROM Diff

Sidebar: ROM TOOLS > ROM Diff

Byte-level diff between an original and a modified ROM, grouped into contiguous changed runs with configurable context, for auditing exactly what a tune changed.

KEY UI ELEMENTS

- ROM A (original) / ROM B (modified) rows; Context rows spinbox (0–32, default 4); "Show only changed regions" checkbox.
- Run Diff / Export Report buttons; live changed-byte count/percentage.

HOW TO USE IT

- 1 Browse to ROM A and ROM B.
- 2 Click Run Diff — differing offsets are grouped into contiguous runs (up to 200 shown), each printed as 16-byte-wide hex rows for both ROMs plus context padding.
- 3 Click Export Report to save a text summary of every run's offset/size.

Patch Export

Sidebar: ROM TOOLS > Patch Export

Generates a distributable patch file capturing only the byte-level differences between an original and modified ROM, in one of four formats.

KEY UI ELEMENTS

- Original ROM / Modified ROM rows; Patch format radios (IPS, Binary Delta, XDF Stub, JSON); Generate Patch button.

HOW TO USE IT

- 1 Browse to load both ROMs.
- 2 Pick a patch format.
- 3 Click Generate Patch and save to the chosen path.

TECHNICAL NOTES

- IPS: classic PATCH...EOF format, runs of consecutive changed bytes (max 65535 bytes/run).
- Binary Delta: compact custom format — record-count header + (offset, new value) records.
- XDF Stub: minimal XDF XML with one table stub per changed byte (capped at 200) — a starting point for a full definition, not a complete one.
- JSON: a plain array of {offset, old, new} records.

ROM History

Sidebar: ROM TOOLS > ROM History

A git-style local snapshot store for ROM binaries, letting a tuner save labelled checkpoints and roll back later.

KEY UI ELEMENTS

- ROM path + Browse + Save Snapshot; Snapshot label field.
- Snapshot table (Timestamp, Label, Size, MD5, File); Rollback to Selected / Diff vs Current / Delete Snapshot / Refresh buttons.

HOW TO USE IT

- 1 Browse to load the ROM to version.
- 2 Enter a label, click Save Snapshot — copies the file into logs/rom_history/ with a timestamped name plus a JSON metadata sidecar.
- 3 Select a row and click Rollback to Selected to copy that snapshot out.
- 4 Click Diff vs Current to compare a snapshot's bytes against the currently loaded ROM.
- 5 Click Delete Snapshot to remove a checkpoint (with confirmation).

ROM Library

Sidebar: ROM TOOLS > ROM Library

A searchable browser over the bundled reference ROM database, letting a tuner locate a stock ROM for a given make/model, open it directly in the hex viewer, or jump to identification.

KEY UI ELEMENTS

- Reload button; live-filter Search field with a result-count label.
- 6-column table (Make, Model, Year, Engine, Size, File); Open in Hex Editor and Identify buttons.

HOW TO USE IT

- 1 On open, the panel indexes every file under the reference ROM database (excluding known non-ROM extensions), capped at 5,000 rows for display.
- 2 Type in Search to live-filter across make, model, and filename.
- 3 Select a row and click Open in Hex Editor to open it in a dedicated, read-focused hex viewer window (separate from the main embedded Hex Editor).
- 4 Select a row and click Identify (currently directs you to the ROM Identify panel rather than running inline).

TECHNICAL NOTES

- Make/Model columns derive from the reference database's folder structure itself (raw/<make>/<model>/...).
- A companion ECU Coverage panel (ROM TOOLS > ECU Coverage) aggregates the same tree into a collapsible Make→Model→file-count index with its own search, useful for checking whether your vehicle's ECU is represented before hunting through the flat file list.

Rev/Speed Limiter

Sidebar: ROM TOOLS > Rev/Speed Limiter

Scans a loaded ROM for byte patterns matching a target RPM or speed value under several common ECU encodings, so a tuner can locate and directly patch the limiter bytes without a known map definition.

KEY UI ELEMENTS

- Two tabs: Rev Limiter and Speed Limiter.
- Search-for-value field (defaults 6500 RPM / 250 km/h) with a $\pm$ tolerance field; Scan ROM button.
- Results table (Offset, Encoding, Decoded, Hex, Context); edit row with New value, Patch Selected, Save ROM.

HOW TO USE IT

- 1 Load a ROM binary (without one, Scan ROM shows demo rows so the UI stays explorable).
- 2 Enter the target value and tolerance, click Scan ROM — every candidate in range is packed through every known encoding and searched for byte matches (capped at 200 results).
- 3 Select a result, enter the new value, click Patch Selected to overwrite those bytes in the in-memory buffer using the same encoding.
- 4 Click Save ROM to write the patched buffer to a new file.

TECHNICAL NOTES

- Rev-limiter encodings searched: 16-bit LE/BE $\div 10$, 16-bit LE/BE $\div 4$, 16-bit LE/BE $\times 1$, 8-bit $\div 100$. Speed-limiter encodings: 8-bit km/h, 16-bit LE/BE km/h, 8-bit mph $\times 1.6$.

DPF/EGR Delete

Sidebar: ROM TOOLS > DPF/EGR Delete

A gated UI for marking emissions-system delete flags for later application to a ROM (DPF, EGR, lambda, NOx/AdBlue, catalyst monitor, EVAP, swirl flap, speed limiter), behind a mandatory legal disclaimer.

KEY UI ELEMENTS

- A legal disclaimer box stating that removing/disabling emissions controls is illegal for road use in most jurisdictions (EU, UK, USA Clean Air Act), with a required acceptance checkbox — every other control in the panel is disabled until this is checked.
- 8 flag cards (DPF Delete, EGR Delete, Lambda/O2 Delete, Speed Limiter Remove, NOx/AdBlue Delete, Catalyst Monitor Off, EVAP System Off, Swirl Flap Delete), each with a checkbox and an Active/Inactive status label.
- Mark Flags in ROM (disabled until disclaimer accepted) / Clear All.

HOW TO USE IT

- 1 Read the disclaimer and check the acceptance box.
- 2 Check whichever flags apply.
- 3 Click Mark Flags in ROM — a confirmation dialog lists the active flags and recommends using ROM Diff and Patch Export to record and distribute the resulting change.

TECHNICAL NOTES

- This is the one panel in the app with an explicit legal-liability gate hard-wired into the UI flow, not just cosmetic.
- "Mark Flags" in the current build is informational only — it does not yet patch ROM bytes itself; it hands off to ROM Diff/Patch Export for the actual edit.

ECU

ECU Flash

Sidebar: ECU > ECU Flash (Ctrl+5)

Writes a tuned ROM binary to the vehicle's ECU over KWP2000 (K-line) or UDS (CAN), or reads the ECU's current ROM back to a file. This is the real panel behind the sidebar entry — a single-page form, not a multi-step wizard.

KEY UI ELEMENTS

- **ROM file** field with Browse button and drag-and-drop support — the tuned binary to write.
- **Backup path (optional)** field — where to save a pre-flash backup.
- **Protocol** dropdown — "KWP2000 (K-Line)" or "UDS (CAN)".
- **Port / channel** and **Baud rate** fields (defaults COM3 / 10400).
- **ECU type (checksum)** dropdown — selects the checksum algorithm for the target ECU family.
- **Security algorithm** dropdown — selects the seed-to-key algorithm used during the security access handshake (e.g. bosch_me7, XOR-constant, and other registered algorithms).
- **Simulation mode** checkbox (on by default) — runs the entire flash sequence against a mock ECU with no hardware required, safe for testing.
- Scrolling output console showing live protocol trace / progress.
- **Flash ROM** and **Read ROM** buttons.

HOW TO USE IT

- 1 Open ECU > ECU Flash from the sidebar (or press Ctrl+5).
- 2 Click Browse next to ROM file and select the tuned .bin to write (or drag-and-drop the file onto the field).
- 3 Optionally set a Backup path so the ECU's current contents are saved before writing.
- 4 Choose Protocol — KWP2000 (K-Line) for a serial/K-line interface, or UDS (CAN) for a CAN interface.
- 5 Set Port/channel and Baud rate to match your flash interface.
- 6 Select the correct ECU type (checksum) and Security algorithm for the vehicle's actual ECU family — a mismatch will fail the checksum or the security handshake.
- 7 Leave Simulation mode checked to do a dry run first, or uncheck it once you're ready to flash real hardware.
- 8 Click **Flash ROM**. Progress streams into the console as percentage complete plus protocol messages. Do not disconnect the interface or lose vehicle power while this is running.
- 9 To pull the ECU's current ROM instead, click **Read ROM**, choose a save location, and wait for the transfer to complete.

TECHNICAL NOTES

- Internally this drives a full KWP2000/UDS flash session — start diagnostic session, security access (seed request → computed key), request download, transfer data in blocks, request transfer exit, ECU reset.
- Security access uses publicly documented seed-to-key algorithms (`ecu/security_access.py`), including a generic XOR-constant algorithm used by some Bosch ME5/ME7 and Siemens ECUs, plus a dedicated Bosch ME7/ME7.5 algorithm.
- A separate `ui/flash_wizard.py` module exists in the codebase implementing a nicer 6-step guided wizard (Protocol → Connection → ROM File → Checksum → Flash → Done, including partial-flash sector skipping, a live sector map, and post-flash readback verification), but it is not wired into the sidebar or any menu — the panel

described above is the one you'll actually reach from ECU > ECU Flash.

TCU / Gearbox

Sidebar: ECU > TCU / Gearbox

Transmission calibration for automatic gearboxes — shift-point tables, torque-converter lockup, line pressure, and torque-management limits.

KEY UI ELEMENTS

- 5 tabs: Upshift Points (km/h) — editable 5×10 grid (gear transitions 1→2...5→6 × throttle 10–100%); Downshift Points (km/h) — matching grid.
- Torque Converter: TCC lockup enable, engage/release speed, minimum lockup gear, slip-speed target, disable-below-coolant-temp, TCC release TPS threshold.
- Line Pressure: single row (bar vs. throttle %), with +10%/-10% quick-scale buttons.
- Torque Limits: shift torque reduction %/duration, max input torque, creep torque, stall torque multiplier.
- Export All (JSON) at the bottom, applying across every tab.

HOW TO USE IT

- 1 Edit any cell/field directly in the relevant tab.
- 2 Use a profile/scale shortcut (Sport ×1.12, Economy ×0.88 for shift points; +10%/-10% for line pressure) if desired.
- 3 Click the tab's Apply button.
- 4 Click Export All (JSON) to persist the full configuration to disk.

TECHNICAL NOTES

- All defaults are hardcoded reference tables rather than pulled from a live ECU, and each tab's Apply button currently only updates a status message — it does not yet push values into the ECU simulator, so treat Apply here as UI-local until wired to a backend.

CAN Bus Analyser

Sidebar: ECU > CAN Bus Analyser

A live CAN frame viewer/sniffer with ID filtering and decoding of a small set of known OBD PIDs.

KEY UI ELEMENTS

- Interface dropdown (Simulator, can0/can1 SocketCAN, PCAN-USB, Kvaser, Vector — only Simulator is currently implemented).
- Filter ID hex entry; Start/Stop/Clear buttons.
- Live stats row (Frames, Rate fps, Filtered count); 5-column frame table (Time, ID, DLC, Data, Decoded).

HOW TO USE IT

- 1 Optionally enter a hex CAN ID in Filter ID to show only matching frames.
- 2 Click Start — generates simulated frames from a fixed pool of IDs at 8–50ms intervals.
- 3 Known IDs decode automatically (e.g. Engine RPM, Vehicle Speed, Throttle Position, Coolant Temp); others show as raw hex.
- 4 Click Stop to halt, Clear to empty the table and reset counters.

TECHNICAL NOTES

- The table keeps a rolling maximum of 500 rows (oldest dropped), inserted newest-first.

HARDWARE

Sensor Cal

Sidebar: [HARDWARE](#) > [Sensor Cal](#)

Guided, step-by-step calibration wizard for the four core analog sensor inputs — MAF, TPS, MAP, and wideband O2 — with a live readout and offset/scale correction.

KEY UI ELEMENTS

- Four sensor-select tabs: Mass Air Flow (MAF), Throttle Position (TPS), Manifold Pressure (MAP), Wideband Lambda (O2).
- Left pane: sensor-specific numbered calibration-steps checklist.
- Correction fields: Offset (add) and Scale (multiply) — default 0.0 / 1.0.
- Buttons: Apply Calibration, Reset to Default.
- Right pane: live numeric readout, Samples counter, running Average, and Sample 5s / Stop buttons.

HOW TO USE IT

- 1 Click the tab for the sensor you want to calibrate.
- 2 Follow the numbered calibration steps shown on the left (these vary per sensor — e.g. verifying idle/WOT reference values).
- 3 Watch the live reading update in the right-hand panel.
- 4 Click Sample 5s to capture a 5-second sample window; the average displays under Avg.
- 5 Enter an Offset and/or Scale to correct the raw reading to a known reference, then click Apply Calibration.
- 6 Use Reset to Default to clear offset/scale back to 0.0 / 1.0.

TECHNICAL NOTES

- Live values are pulled from the active ECU connection (simulator or live) on a background poll every 0.2s.
- Reference ranges shown in the step text: MAF idle 2–8 g/s; TPS 0–2% closed / 98–100% full; MAP -0.6 to -0.9 bar vacuum at idle; wideband 0.5V = lambda 1.0 (14.7 AFR).
- Apply Calibration records the offset/scale for reference — it does not write anything to the ECU itself.

VIN Decoder

Sidebar: [HARDWARE](#) > [VIN Decoder](#)

Decodes a 17-character VIN into manufacturer, model year, and structural segments (WMI/VDS/VIS), and saves the result to a vehicle profile.

KEY UI ELEMENTS

- 17-character monospace VIN entry field with Decode, Read from ECU, and Paste buttons.
- VIN breakdown canvas — 17 colour-coded character cells (WMI/VDS/VIS) with position numbers.
- Decoded field list: Full VIN, Manufacturer, Model Year, WMI, VDS, VIS, Assembly Plant, Production Sequence, Check Digit.
- Save to Vehicle Profile button.

HOW TO USE IT

- 1 Type or paste a 17-character VIN, or click Read from ECU to pull one from the connected controller.
- 2 Press Enter or click Decode.

- 3 Review the colour-coded breakdown and the decoded field list.
- 4 Click Save to Vehicle Profile and choose a .json location to persist it.

TECHNICAL NOTES

- Decoding is entirely offline using a built-in WMI prefix table covering roughly a dozen manufacturers (GM, Ford, Chrysler, Honda, BMW, Mercedes, Renault/PSA, Jaguar, Nissan, Toyota, Ferrari) — unmatched prefixes report "Unknown manufacturer."
- Model year decodes from VIN position 9 via a year-code table covering 1980–2009 and 2010–2024.
- Read from ECU currently always returns a fixed demo VIN rather than a real hardware read.

Wiring View

Sidebar: HARDWARE > Wiring View

Reference viewer for ECU/OBD connector pinouts across a small library of common connector types.

KEY UI ELEMENTS

- ECU/Connector dropdown: Generic OBD-II DLC, Subaru EJ20 ECU, VAG MED17 ECU.
- Search pin/function text filter.
- Left: colour-coded connector diagram (numbered pin ovals) with a detail label.
- Right: pin table — columns Pin #, Function, Notes.

HOW TO USE IT

- 1 Select a connector/ECU type from the dropdown.
- 2 Optionally type a search term to filter the pin table.
- 3 Click a row in the table, or a pin in the diagram, to highlight it and show its colour code and function.

TECHNICAL NOTES

- Pinout data is built in for three connectors: 16-pin Generic OBD-II DLC (SAE J1850, ISO 9141-2 K/L-line, CAN High/Low, battery), 22-pin Subaru EJ20, and 22-pin VAG MED17.
- Each pin is colour-coded by function category (fuel, sensors, ground, etc.) consistently between the table and diagram.

CALC

Injector Calc

Sidebar: CALC > Injector Calc

Three linked tools for fuel-injector sizing: a target-power sizing calculator, a duty-cycle calculator, and an editable dead-time/offset table with a plotted dead-time curve.

KEY UI ELEMENTS

- Sizing calculator: Target power (WHP), Cylinders, BSFC (lb/hr/hp), Max duty cycle (%), Fuel type (Gasoline 91/93, E85, E10, Methanol), Target WOT AFR, Safety headroom (%) — Calculate Injector Size button.
- Duty-cycle calculator: Engine RPM, Injector pulse width (ms), Injectors per bank, Injection sequence — Calculate Duty Cycle button, colour-coded result (OK ≤80%, WARNING ≤90%, OVER LIMIT >90%).
- Dead-time table: Preset dropdown (8 built-in injectors incl. Bosch 630cc/1000cc, ID1050x/1300x/2000, Siemens Dekka 60lb, Injector Dynamics 725cc), directly-editable 12-row Voltage/Dead-Time/Offset table, Export CSV.
- Live dead-time curve chart (voltage vs. dead-time).

HOW TO USE IT

- 1 For sizing: enter target WHP, cylinder count, BSFC, max duty %, safety headroom %, fuel type, and target WOT AFR, then click Calculate Injector Size to see required injector size in lb/hr and cc/min, estimated pulse width at 6500 RPM, and the nearest standard preset.
- 2 For duty cycle: enter RPM and pulse width, click Calculate Duty Cycle.
- 3 For dead time: pick a preset to auto-fill scaled values, or edit any cell directly — the curve redraws automatically. Click Export CSV to save.

TECHNICAL NOTES

- Fuel-mass-flow: $\text{fuel_flow_total} = \text{HP} \times \text{BSFC}$, split across cylinders, then scaled by target duty cycle and safety headroom.
- cc/min conversion uses per-fuel density/stoich/LHV tables (e.g. Gasoline 91oct = 0.740 g/mL, stoich 14.7; E85 = 0.785 g/mL, stoich 9.76).
- Duty cycle assumes a 4-stroke cycle: $\text{cycle_time_ms} = (60/\text{RPM}) \times 2 \times 1000$; $\text{duty\%} = \text{pulse_width} / \text{cycle_time} \times 100$.

Turbo Calc

Sidebar: CALC > Turbo Calc

Estimates required compressor pressure ratio and mass airflow for a target boost level, plots the operating point on a compressor map, and overlays a selected turbo's efficiency islands, surge/choke lines, and speed lines.

KEY UI ELEMENTS

- Engine Specifications: Displacement (L), Cylinders, Volumetric efficiency (%), Charge air temp / IAT (°C), Altitude (m).
- Boost Target: Boost target (bar gauge), RPM range low/high, WOT AFR.
- Turbo Preset dropdown: GT2860RS, GT3071R, GT35R, Precision 6266, Garrett G25-660, Garrett G35-900.
- Calculate & Plot Operating Point button; colour-tagged results text.
- Compressor map canvas: Corrected Mass Flow (x) vs. Pressure Ratio (y), efficiency islands, Surge/Choke lines, turbo speed lines, and a crosshair marking the calculated operating point.

HOW TO USE IT

- 1 Enter engine displacement, cylinders, volumetric efficiency, charge air temp, and altitude.

- 2 Enter boost target, RPM range, and target WOT AFR.
- 3 Select a turbo preset (redraws the map immediately).
- 4 Click Calculate & Plot Operating Point.
- 5 Read pressure ratio, mass flow at high/low RPM, compressor outlet temperature rise, estimated power capacity, and any flow-margin warning.
- 6 Check the compressor map to see where the operating point falls relative to efficiency islands, surge, and choke.

TECHNICAL NOTES

- Ambient pressure correction: $p_{\text{amb}} = 101.325 \times \exp(-\text{altitude}/8500)$ kPa.
- Compressor outlet temperature uses an ideal adiabatic rise divided by a fixed 0.72 compressor efficiency.
- Estimated power uses a fixed BSFC assumption (0.45 lb/hr/hp) — treat the HP estimate as a ballpark, not a dyno-verified figure.

REPORTS

Tune Sheet PDF

Sidebar: REPORTS > Tune Sheet PDF

Generates a formatted customer-handover document (PDF/HTML/print) summarizing every changed byte/cell between a stock and tuned ROM, with customer/vehicle metadata.

KEY UI ELEMENTS

- ROM Files: Stock ROM / Tuned ROM Browse pickers.
- Report Details: Customer name, Vehicle, VIN, ECU part number, Tuner/garage, Tune date, Notes.
- Include-in-report checkboxes (diff table, fuel/ignition summaries, boost tables, safety limits, dyno results, garage branding).
- Diff preview — scrollable monospace table: offset, stock value, tuned value, colour-coded delta, description.
- Buttons: Analyse Diff, Export PDF, Export HTML, Print.

HOW TO USE IT

- 1 Browse to select both the Stock ROM and Tuned ROM files (or skip to generate a demo diff).
- 2 Fill in customer, vehicle, VIN, ECU part number, tuner/garage, date, and notes.
- 3 Tick which sections to include.
- 4 Click Analyse Diff to byte-compare the two ROMs and populate the preview.
- 5 Click Export PDF, Export HTML, or Print to produce the final document.

TECHNICAL NOTES

- The diff is a raw byte-by-byte comparison over the shorter file's length, with a size-mismatch warning if they differ.
- PDF export is built with ReportLab (title page, metadata table, coloured changed-bytes table capped at 200 rows); HTML export caps at 500 rows.

WORKFLOW

Fault Codes

Sidebar: *WORKFLOW > Fault Codes (Ctrl+6)*

Scans the ECU for Diagnostic Trouble Codes (DTCs) and shows a live sensor-health panel side by side, so you can correlate a fault against current readings.

KEY UI ELEMENTS

- Scan for Faults and Clear All Codes buttons; Live sensor monitor checkbox.
- Split layout: left is a DTC table (Code, Description, Category, Severity, Status), colour-coded by severity; right is Live Sensor Health with threshold-coloured indicators per channel.
- Detail strip showing the full description of the selected code; Export Report button.

HOW TO USE IT

- 1 Open WORKFLOW > Fault Codes (or Ctrl+6) — it scans automatically on open.
- 2 Click Scan for Faults at any time to re-read current DTCs.
- 3 Click a DTC row to see its full description in the detail strip.
- 4 Watch Live Sensor Health — each channel's status dot changes colour if a value crosses a warning/critical threshold.
- 5 Once the issue is fixed, click Clear All Codes to reset the ECU's fault memory.
- 6 Click Export Report to save the current scan for the customer or job record.

TECHNICAL NOTES

- This is a separate, self-contained implementation from Fault Codes+ below — both are real, independent, reachable panels.

Fault Codes+

Sidebar: *WORKFLOW > Fault Codes+*

A more advanced 4-tab DTC toolkit: live scan/clear, a full offline DTC reference library, freeze-frame capture, and a compiled printable diagnostic report.

KEY UI ELEMENTS

- Live Faults tab: Scan ECU / Clear All DTCs / Freeze Frame / Add to Report buttons; active-fault table (Code, Description, Status, Category) with a right-click menu (View Details, Capture Freeze Frame, Add to Report, Mark as Known/Ignore); live 8-channel sensor readout with Start/Stop Live (2s poll).
- DTC Library tab: search box over the full code library; detail pane (description, common causes, recommended fixes, related PIDs).
- Freeze Frames tab: Capture Now / Clear All / Export CSV; table of saved frames with a full-snapshot detail pane.
- Fault Report tab: Shop Name, Technician, Vehicle, VIN/Reg, Mileage, Date fields; Generate PDF / Export Text / Export CSV; live text preview.

HOW TO USE IT

- 1 On Live Faults, click Scan ECU to pull codes.
- 2 Right-click a code for View Details, Capture Freeze Frame, or Add to Report.
- 3 Click Start Live to stream the 8 live sensor gauges every 2 seconds.

- 4 Switch to DTC Library to search the full reference independent of a live scan.
- 5 On Freeze Frames, click Capture Now for a manual snapshot; Export CSV to save them all.
- 6 On Fault Report, fill in shop/technician/vehicle details and click Generate PDF — the report includes every code added via "Add to Report" (or all currently active codes if none were explicitly added).

TECHNICAL NOTES

- The DTC library is a built-in dictionary of roughly 55 real-world P-codes (P0001–P0606), each with causes, fixes, and related PIDs — entirely offline, no external lookup.

Remote Tune

Sidebar: *WORKFLOW > Remote Tune*

Lets a tuner ("host") broadcast their live ECU sensor stream to a remote expert ("guest") via a shared JSON session file, with a lightweight chat/notes channel — no server required, just a shared file path.

KEY UI ELEMENTS

- Mode radios: Host (share my data) / Guest (view remote data).
- Host: Start Session / Stop, session file path + Copy Path, live gauge grid (RPM, Boost, TPS, MAF, CLT, STFT, LTFT, Knock, O2V), Session Notes log + Send.
- Guest: Open Session File..., identical gauge grid, Last update timestamp + lag indicator, read-only Host Notes log.

HOW TO USE IT

- 1 Host: select Host mode, click Start Session — a JSON file is created under logs/remote_sessions/ and updates every 250ms.
- 2 Click Copy Path and share it with the remote tuner via any file-sharing channel (network drive, cloud sync folder, email).
- 3 Type notes and click Send to append them, visible to the guest.
- 4 Guest: select Guest mode, click Open Session File... and browse to the shared path — the gauge grid updates automatically as the file changes.
- 5 Host clicks Stop to end the session (deletes the session file).

TECHNICAL NOTES

- Architecture is entirely file-based — no network socket. Host writes session ID, timestamp, notes, and live fields every 0.25s; guest polls the same file every 0.3s and only re-renders when the timestamp changes.

Marketplace

Sidebar: *WORKFLOW > Marketplace*

Community tune-sharing hub — browse, search, filter, download, review, and submit shared ECU tunes across a catalog of vehicle/engine combinations.

KEY UI ELEMENTS

- View tabs: Browse All, My Downloads, Favourites; Submit a Tune... button.
- Editor's Picks card strip; search box + Sort dropdown (Downloads/Rating/Newest/A-Z); tag-chip filter bar (Stage 1/2/3, Street, Track, Rally, Drift, Diesel, E85, etc.); cascading Make → Model → Engine dropdowns.
- Left: scrollable tune card list. Right: detail pane — Download Tune, ♥ Favourite, Import into Map Editor, Flag as Unsafe buttons, meta grid, a simulated dyno chart, Similar Tunes, and Community Reviews with a star-rating review form.

HOW TO USE IT

- 1 Browse or search/filter the tune list; click a card to open its detail pane.
- 2 Click Download Tune and choose a save location.
- 3 Click ♥ Favourite to save it to your Favourites tab.
- 4 Click Import into Map Editor for guidance on loading the downloaded tune (Map Editor → Load ROM → ROM Diff against your stock ROM).
- 5 Scroll to Write a Review to rate and comment on a tune.
- 6 Click Submit a Tune... to open a submission dialog (title, vehicle, ECU type, power gain, tags, description, optional file attachment).

TECHNICAL NOTES

- The catalog is a static, built-in list of roughly 20 sample tunes spanning many real platforms (VW/Audi EA888, Subaru EJ20/EJ257, Honda K20C1, BMW N54/B58/B48, Toyota 1JZ/2JZ/2ZZ, Mitsubishi 4G63/4B11T, LS3, Coyote 5.0, diesel TDIs).
- Favourites, downloads, and added reviews are in-memory only for the current session — they are not persisted to disk. Submit a Tune and Flag as Unsafe are front-end confirmations only, with no backend upload/moderation queue behind them yet.

Customers

Sidebar: WORKFLOW > Customers — opens as its own window

Central customer relationship + vehicle registry: add/edit/delete customers, manage each customer's vehicle fleet, and jump directly into job creation for a specific vehicle.

KEY UI ELEMENTS

- Header search box; New Customer / Edit / Delete buttons.
- Customer table (Name, Phone, Email, Address, Vehicles count).
- Vehicles section for the selected customer: Add Vehicle / Edit Vehicle / Remove / New Job for this Vehicle; vehicle table (Reg Plate, Make, Model, Year, Engine, VIN).

HOW TO USE IT

- 1 Click New Customer, fill in Name (required) plus Phone/Email/Address/Notes, Save.
- 2 Select a customer to load their vehicles below.
- 3 Click Add Vehicle and fill in reg/make/model/year/engine/VIN/notes — if the make/model matches the bundled ROM database, a confirmation dialog shows the match before saving.
- 4 Select a vehicle and click New Job for this Vehicle to jump straight into job creation with customer/vehicle pre-filled.

TECHNICAL NOTES

- Deleting a customer cascades and deletes their vehicles too — the app warns before this happens.

Jobs

Sidebar: WORKFLOW > Jobs — opens as its own window

Work-order / job-card system tracking each tuning engagement per customer/vehicle, including service type, technician, pricing, payment status, mandatory customer consent, and a full audit trail per job.

KEY UI ELEMENTS

- Status filter tabs: All Jobs, Booked, In Progress, Complete, Invoiced (colour-coded).

- Job table (#, Customer, Vehicle, Title, Service, Status, Tech, Price, Paid, Date).
- New Job / View Details / Launch Workflow / Update Status / Mark Paid / Delete Job / Refresh buttons.
- New Job dialog: Customer* (auto-loads vehicles), Vehicle, Job Title*, Service Type dropdown, Technician, Price, Notes, and a mandatory consent checkbox.

HOW TO USE IT

- 1 Click New Job, select a customer (auto-loads their vehicles), fill in title, service type, technician, and price.
- 2 Tick the consent checkbox — required; saving is blocked without it.
- 3 Click Create Job — if the service type maps to a known tool workflow, you're offered to launch it immediately.
- 4 Use the status tabs or Update Status to progress Booked → In Progress → Complete → Invoiced — every change logs to the audit trail.
- 5 Click Mark Paid when payment is received.
- 6 Click View Details for the full record and audit log, or Launch Workflow to jump directly into the relevant tuning tool.

TECHNICAL NOTES

- Every create/status-change/payment action is permanently logged to a per-job audit trail (Time, Action, Detail, Technician).

Customer Portal

Reached via BUSINESS > Business Hub > "Customer Portal" tab — not a standalone sidebar entry

Runs a local, read-only web server (no external dependency) that lets a shop hand a customer a private link showing their vehicles and job/tune history, without exposing the desktop app, database, or other customers' data.

KEY UI ELEMENTS

- Port entry field, Start Server / Stop Server buttons, status label.
- Customer Links table (Customer, Portal Link) with Refresh; Open Selected in Browser and Copy Link buttons.

HOW TO USE IT

- 1 Set a port (default 8765) and click Start Server.
- 2 The link list auto-populates: one unique token-based URL per customer.
- 3 Select a customer and click Open Selected in Browser to preview their page, or Copy Link to hand it to them (invoice, text, etc.).
- 4 Click Stop Server when done.

TECHNICAL NOTES

- The portal page shows a Your Vehicles table and a Service & Tune History table — nothing else.
- Each customer gets a random, unguessable token stored locally, so links can't be used to enumerate other customers. The server binds only to 127.0.0.1 by design — it is a local shop-network/kiosk view, never exposed to the internet.

BUSINESS

Business Hub

Sidebar: BUSINESS > Business Hub

A six-tab back-office suite covering tune version tagging, invoicing, a global audit log, cloud/remote sync, ROM digital signatures, and the embedded Customer Portal controls.

KEY UI ELEMENTS

- Tune Versions tab: Tag Current Session / Export List CSV / Refresh; table of Version, Date, Customer, Vehicle, Role, Hash, Notes.
- Invoices tab: customer/vehicle/date/tax-rate fields, a line-items table with 6 built-in service presets, running Subtotal/Tax/Total, Preview / Generate PDF / Save Invoice.
- Audit Log tab: global append-only event log, event-type filter, search, Export CSV, Clear Old (>30d).
- Cloud Sync tab: point at any locally-synced folder (Dropbox/OneDrive/Google Drive/network share); file-type checkboxes; Sync Now / Save Settings.
- Digital Signature tab: sign a ROM with a SHA-256 sidecar, Verify Signature, and a Signature History table.
- Customer Portal tab: embeds the Customer Portal panel directly (see WORKFLOW chapter).

HOW TO USE IT

- 1** Tune Versions: click Tag Current Session, fill in customer/vehicle/notes/role, save — appends a v1, v2... entry with a SHA-256 marker hash.
- 2** Invoices: fill in customer/vehicle details, add line items (or use a Preset... like "ECU Remap Stage 1 £350"), then Generate PDF or Save Invoice.
- 3** Audit Log: filter by event type or search, Export CSV for records, or Clear Old to purge entries older than 30 days.
- 4** Cloud Sync: browse to a sync folder, tick which file types to include, click Sync Now — copies matching files into a timestamped subfolder there.
- 5** Digital Signature: select a ROM, enter signer name and role, click Sign ROM — writes a .sig sidecar; click Verify Signature later to detect tampering.

TECHNICAL NOTES

- Important correction: despite the role dropdown (Technician / Senior Tuner / Admin) appearing throughout this hub, it is purely a record-keeping label attached to log/report entries — there is no PIN or permission enforcement tied to it in this panel. The Technician Accounts panel (below) has its own, more detailed permission matrix, which is likewise an informational reference display, not an enforced technical control.
- Cloud Sync explicitly does not transmit anything to the internet itself — the "cloud" aspect is entirely delegated to whatever sync client watches the folder you choose. The Sync Schedule radio (Manual/On Save/Hourly) is label-only; no scheduler is actually wired up to trigger it automatically.
- The audit log is capped at the most recent 2000 events on disk (500 shown in the table) and is fed by tune-version tagging, invoice creation, and ROM signing throughout the app.

Technician Accounts

Sidebar: BUSINESS > Technician Accounts

Multi-user account management for workshop staff with a five-tier role hierarchy and a reference permissions matrix; supports switching the active session identity used for attribution elsewhere.

KEY UI ELEMENTS

- Active session bar with Switch User button.
- Accounts table (Username, Display Name, Role, Status); Add Account / Edit Role / Reset Password / Deactivate / Delete Account.
- Role Permissions matrix — a static reference table.

HOW TO USE IT

- 1 Click Add Account, enter username/display name/password, choose a role, Create.
- 2 Select an account and use Edit Role, Reset Password, Deactivate, or Delete Account (the last two are blocked for the currently logged-in account).
- 3 Click Switch User and enter matching credentials to change the active session identity shown elsewhere in the app.

TECHNICAL NOTES

- Roles: Owner, Senior Tuner, Tuner, Technician, Viewer.
- The displayed permission matrix (edit maps, flash ECU, access ROM tools, view live data, manage customers/jobs, add/remove accounts, export to Marketplace) is a static reference only — no code path in the app actually gates feature access against it. Treat it as documented shop policy for staff to follow, not a technical restriction the app enforces.
- Accounts persist locally; passwords are stored as unsalted SHA-256 hashes. A default admin/admin123 Owner account seeds a fresh install — change this before relying on accounts for anything sensitive.

Cloud Backup

Sidebar: BUSINESS > Cloud Backup

Automated (or manual) backup of ROMs, datalogs, and the database to a local folder and/or a mapped cloud/network path.

KEY UI ELEMENTS

- Local backup folder + Browse; Cloud/network path free-text field.
- Schedule: enable-automatic-backups checkbox, frequency dropdown (After every flash / Hourly / Daily / Weekly), include-in-backup checkboxes.
- Backup History table; Backup Now / Save Settings / Open Backup Folder.

HOW TO USE IT

- 1 Browse to a local backup folder and/or type a cloud/network path.
- 2 Tick Enable automatic backups, pick a frequency and what to include, Save Settings.
- 3 Click Backup Now to run an immediate backup on a background thread.
- 4 Click Open Backup Folder to view the destination in Explorer.

TECHNICAL NOTES

- The current backup routine is a demonstration implementation: it copies the app's local JSON data files into a timestamped destination subfolder rather than walking the real ROM/datalog/database locations the checkboxes imply. The "automatic" schedule options are UI state only — nothing triggers a backup on the configured cadence yet. Do not rely on this as your only ROM backup mechanism; use ROM History for that.

Reminders

Sidebar: BUSINESS > Reminders

Tracks scheduled customer follow-ups (post-tune checks, service intervals, re-map reviews) by due date, so a shop never misses a revisit.

KEY UI ELEMENTS

- Four summary cards: Overdue, Due this week, Upcoming, Completed.
- Filter radios; reminders table (Customer, Vehicle, Type, Due Date, Miles Due, Notes, Status).
- Add Reminder / Mark Done / Edit / Delete.

HOW TO USE IT

- 1 Click Add Reminder, fill in customer, vehicle, type (Post-tune check, 12-month re-map, Stage upgrade review, Oil change, Boost check, Lambda sensor check, Gearbox service, Custom), due date, miles due, notes, Save.
- 2 Use the filter radios or summary cards to focus on a subset.
- 3 Select a reminder and click Mark Done, Edit, or Delete.

TECHNICAL NOTES

- Status is computed live on every refresh: overdue if due date has passed, "this week" if due within 7 days, otherwise upcoming.

Setup Wizard

Sidebar: BUSINESS > Setup Wizard — also auto-launches on first run

A five-step first-run wizard introducing DORA, connecting to an ECU (or simulator), optionally loading a starter ROM, choosing a primary workflow, and summarising the setup before handing off to the main app.

KEY UI ELEMENTS

- 5 step-progress dots: Welcome, Connect, ROM, Workflow, Ready.
- Connect step: Use Simulator or Connect to ECU (reveals Port/Baud dropdowns + Test Connection).
- ROM step: Browse for ROM... (accepts .bin/.rom/.ecu/.hex), optional .xdf definition load, or Skip.
- Workflow step: 2x2 grid of starting-point cards (Quick Tune, Data Analysis, Dyno Session, ROM Workshop).
- Ready step: summary + "Don't show this wizard on startup" checkbox (checked by default).

HOW TO USE IT

- 1 Click Next through Welcome.
- 2 On Connect, choose Simulator or a real ECU connection with Port/Baud.
- 3 On ROM, optionally load a starter ROM (or skip).
- 4 On Workflow, pick the card matching what you want to do first.
- 5 Review the Ready summary and click Finish.

TECHNICAL NOTES

- First-run state only persists (so the wizard won't reappear) if the "don't show again" box remains checked at Finish.

SYSTEM

Def Builder

Sidebar: SYSTEM > Def Builder

Builds and edits ECU XML table definitions (compatible with DORA's ROM parser) by visually selecting byte regions in a hex viewer and specifying table geometry, type, and scaling — for adding support for a ROM that isn't already in the reference database.

KEY UI ELEMENTS

- Toolbar: Load ROM, Import XML, Export XML.
- Left: hex viewer (Go to offset, selection status, 16-bytes/row hex+ASCII grid).
- Right: Table Definition form (name, offset, rows, cols, scaling, units, data type — 12 options covering u/int8/16/32 in both endiannesses); Add Table, From Selection, Preview.
- Defined Tables list + Remove Selected; a Value Preview panel of parsed, scaled table values.

HOW TO USE IT

- 1 Click Load ROM and select a binary — it renders in the hex viewer.
- 2 Click a byte to set the selection start; Shift-click to extend it.
- 3 Click From Selection to auto-fill the Offset field from the selection.
- 4 Fill in name, rows, cols, scaling, units, and data type.
- 5 Click Preview to sanity-check the parsed values without committing.
- 6 Click Add Table to commit it to the Defined Tables list.
- 7 Click Export XML once all tables are defined.

TECHNICAL NOTES

- Exported XML uses a `<RomID>/<tables>/<table>` structure with attributes name, offset, sizey, sizex, storagetype, scaling, units — exactly what DORA's ROM parser expects, so a definition built here plugs straight into the Map Editor.

Settings

Sidebar: SYSTEM > Settings

Application-level configuration — serial connection defaults, logging verbosity, and licence key — persisted directly to the app's .env file.

KEY UI ELEMENTS

- OBD Interface: Serial Port combobox + Detect button, Baud Rate dropdown.
- Application: Log Level dropdown.
- Licence: Licence Key field (masked, with a Show toggle).
- Save Settings / Cancel.

HOW TO USE IT

- 1 Click Detect to auto-populate the Serial Port dropdown, or type one manually.
- 2 Choose a Baud Rate matching your OBD adapter.
- 3 Choose a Log Level.

- 4 Enter your Licence Key.
- 5 Click Save Settings — writes directly into the .env file; changes take effect after restart.

TECHNICAL NOTES

- This is the graphical equivalent of editing .env by hand (see Part I Configuration) — it updates the same OBD_PORT / OBD_BAUD_RATE / LOG_LEVEL / LICENCE_KEY values.

Command Palette

Global: Ctrl+K — not a sidebar item

Fuzzy-search quick-navigation overlay across every section of the app, so you can jump anywhere by typing a few characters instead of hunting through the sidebar.

KEY UI ELEMENTS

- Semi-transparent veil with a centred search box; up to 8 live-filtered result rows; footer hint (↑↓ navigate · Enter select · Esc close).

HOW TO USE IT

- 1 Press Ctrl+K.
- 2 Type part of a section's name or an associated keyword (e.g. "boost", "afr", "checksum", "vin").
- 3 Use ↑/↓ or the mouse to pick a row, Enter (or click) to navigate.
- 4 Esc or clicking the veil closes without navigating.

TECHNICAL NOTES

- Covers all 31 destinations across the whole app. Matching favours an exact substring hit (scored by position), falling back to an in-order fuzzy character match; results are ranked and capped at 8.

Notification Center

Global: header bell icon — not a sidebar item

A slide-in panel listing recent alerts, warnings, info, and success messages generated anywhere in the app.

KEY UI ELEMENTS

- Header: title, X close, Clear all.
- Scrollable card list — coloured left strip by severity, icon + title, timestamp, optional body text, per-item dismiss.

HOW TO USE IT

- 1 Click the bell icon to open the panel.
- 2 Review notifications, colour-coded by severity (info/warn/error/success).
- 3 Dismiss individually with each card's X, or Clear all.
- 4 Click elsewhere or press Esc to close without clearing.

TECHNICAL NOTES

- The panel itself has no persistence or subscription mechanism — whatever component opens it supplies the current notification list at that moment.

Troubleshooting

Symptom	Likely cause / fix
No ports listed in the connection dialog	Adapter driver isn't installed, or the adapter isn't plugged in / paired. Reconnect it and click Refresh.
"Could not connect" on the selected port	Check the ignition is ON, the correct port is selected, and try a different baud rate (9600 / 38400 / 57600 / 115200).
Gauges show no movement	Confirm the header pill and mode badge read LIVE, not SIM — if it still reads Simulator, the connect step didn't complete.
AI Analysis panel is greyed out / errors immediately	No LICENCE_KEY or ANTHROPIC_API_KEY is configured — see the Configuration section in Part I.
A map change is refused with a BLOCKED message	The Safety Guard rejected it — see Part II. Reduce the size or scope of the change.
ECU Flash reports a negative response / security access denied	Confirm the ECU type (checksum) and Security algorithm dropdowns match your vehicle's actual ECU family — a mismatch fails the seed/key handshake.
J2534 device not detected	Install the vendor's Windows driver/DLL first (it registers the device under SOFTWARE\PassThruSupport.04.04 in the registry) — DORA only auto-detects devices that are already registered.
Unsure if it's a software or hardware issue	Switch to Simulator mode first to confirm the app and panels work, then isolate the hardware side.

Keyboard Shortcuts Quick Reference

Shortcut	Action
Ctrl + K	Command palette — search / jump to any panel
Ctrl + 1 – Ctrl + 6	Jump to Live Monitor / Map Editor / Dyno / AI / Flash / Fault Codes
Ctrl + 0	Home
K	Arm trigger logging (from Live Monitor)
C	Open Gauge Editor (from Live Monitor)